

5.4 分支限界

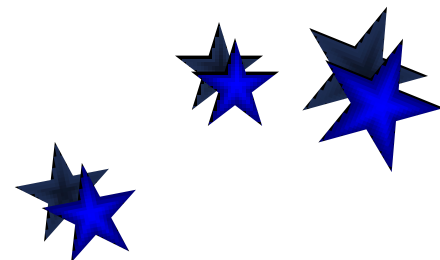
货郎问题的定义

- 输入

有穷个城市的集合 $C=\{c_1, c_2, \dots, c_n\}$,
距离 $d(c_i, c_j)=d(c_j, c_i) \in \mathbb{Z}^+$, $1 \leq i < j \leq n$

- 解: $1, 2, \dots, n$ 的排列 $k_1, k_2, \dots, k_n$ 使得:

$$\min \left\{ \sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1}) \right\}$$



5.4 分支限界

算法设计

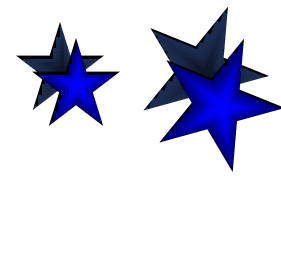
解向量为 $\langle 1, i_1, i_2, \dots, i_{n-1} \rangle$, 其中 $i_1, i_2, \dots, i_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列.

搜索空间为排列树, 结点 $\langle i_1, i_2, \dots, i_k \rangle$ 表示得到 k 步路线.

约束条件: 令 $B = \{i_1, i_2, \dots, i_k\}$, 则

$$i_{k+1} \in \{2, \dots, n\} - B$$

即每个结点只能访问一次.



5.4 分支限界

代价函数与界

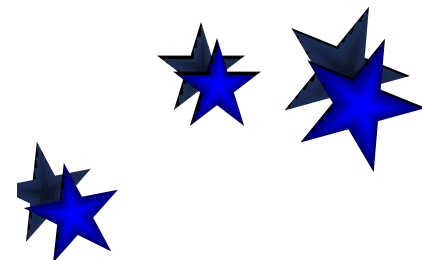
界：当前得到的最短巡回路线长度

代价函数：设顶点 c_i 出发的最短边长度为 l_i , d_j 为选定巡回路线中第 j 段的长度

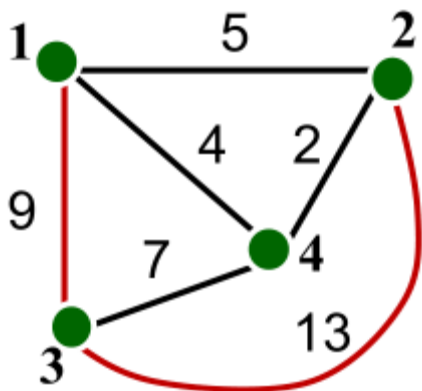
$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

已走过
路径长

剩余长
度下界



5.4 分支限界



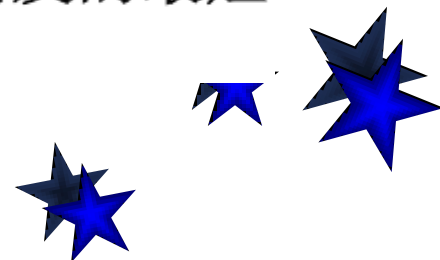
$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j}$$

部分路线<1,3,2>

$$L=9+13+2+2=26$$

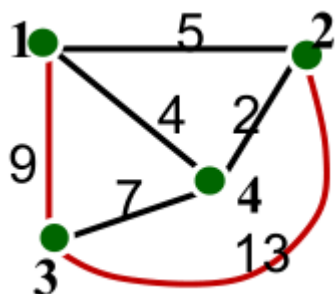
9+13为走过的路径长度

后两项分别为从结点2及4出发的最短边长

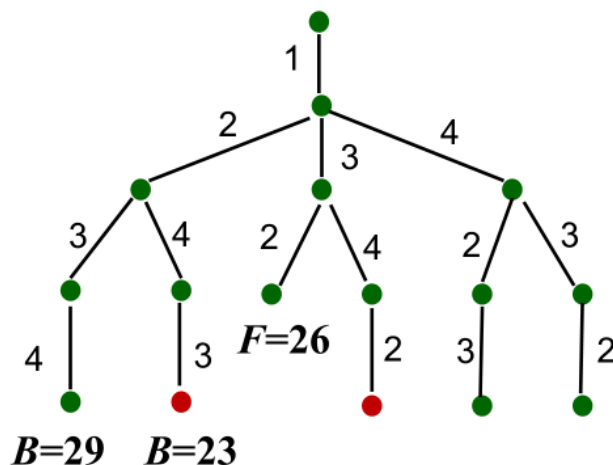


5.4 分支限界

实例运行



搜索树



深度优先遍历搜索树

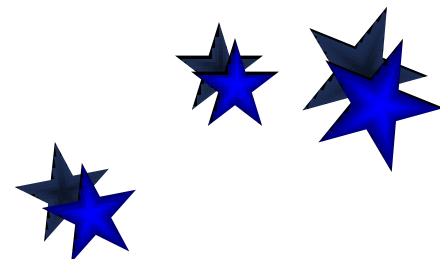
- 第一个界: $\langle 1, 2, 3, 4 \rangle$, 长度为 29
- 第二个界: $\langle 1, 2, 4, 3 \rangle$, 长度为 23
- 结点 $\langle 1, 3, 2 \rangle$: 代价函数值 $26 > 23$, 不再搜索, 返回 $\langle 1, 3 \rangle$, 右子树向下
- 结点 $\langle 1, 3, 4 \rangle$, 代价函数值 $9 + 7 + 2 + 2 = 20$, 继续, 得到可行解 $\langle 1, 3, 4, 2 \rangle$, 长度 23.
- 回溯到结点 $\langle 1 \rangle$, 沿 $\langle 1, 4 \rangle$ 向下...

最优解 $\langle 1, 2, 4, 3 \rangle$ 或 $\langle 1, 3, 4, 2 \rangle$, 长度 23

5.4 分支限界

算法分析

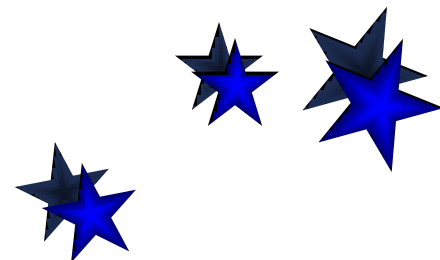
- 搜索树的树叶个数: $O((n-1)!)$,
每片树叶对应 1 条路径, 每条路径有 n 个结点.
- 每个结点代价函数计算时间 $O(1)$,
每条路径的计算时间为 $O(n)$
- 最坏情况下算法的时间 $O(n!)$



5.4 分支限界

小结

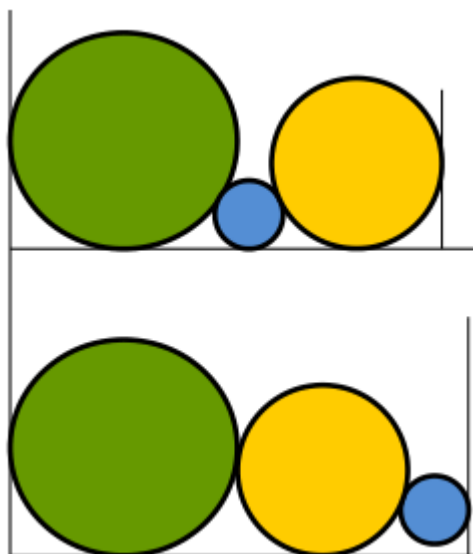
- 货郎问题的分支限界算法：
约束条件：只能选没有走过的结点
代价函数：走过长度+后续长度的
下界
- 时间复杂度： $O(n!)$



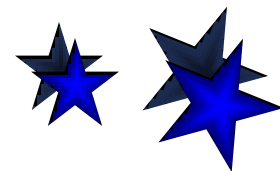
5.4 分支限界

圆排列问题

给定 n 个圆的半径序列, 各圆与底线相切排列, 假定每个圆占大于1的长度, 求具有最小长度 l_n 的圆的排列顺序.



三个圆,半径给定
两种排列具有不同排列长度, 第一种方式具有更小的排列长度



5.4 分支限界

算法设计：分支限界

解： $\langle i_1, i_2, \dots, i_n \rangle$ 为 $1, 2, \dots, n$ 的排列

搜索空间为排列树

部分解向量 $\langle i_1, i_2, \dots, i_k \rangle$ ：表示前 k 个圆已排好. 令

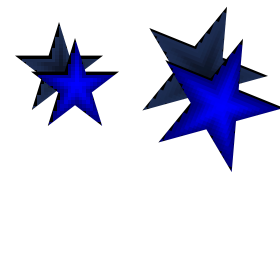
$$B = \{i_1, i_2, \dots, i_k\}$$

下一个圆选择 i_{k+1}

约束条件： $i_{k+1} \in \{1, 2, \dots, n\} - B$

界：当前得到的最小圆排列长度

3



5.4 分支限界

符号说明

k : 算法已经选择了第1— k 个圆

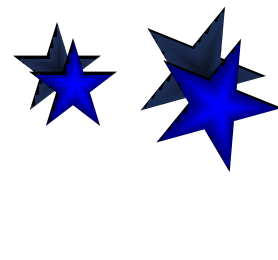
r_k : 第 k 个圆的半径

d_k : 第 $k-1$ 个圆到第 k 个圆的圆心水平距离, $k>1$

x_k : 第 k 个圆的圆心坐标, 规定 $x_1=0$,

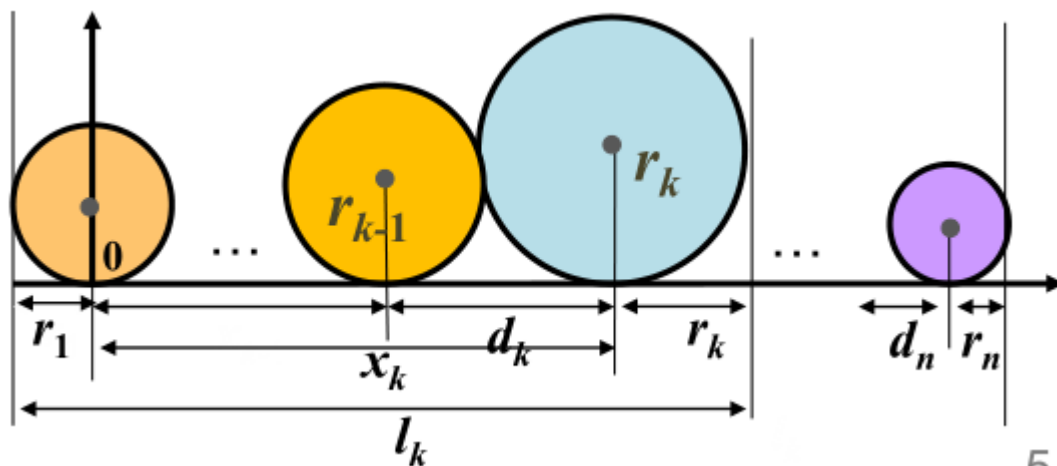
l_k : 第 1— k 个圆的排列长度

L_k : 放好 1— k 个圆以后, 对应结点的代价函数值 $L_k \leq l_n$



5.4 分支限界

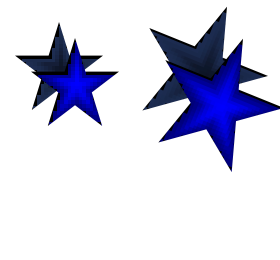
圆排列长度估计



$$x_k = x_{k-1} + d_k$$

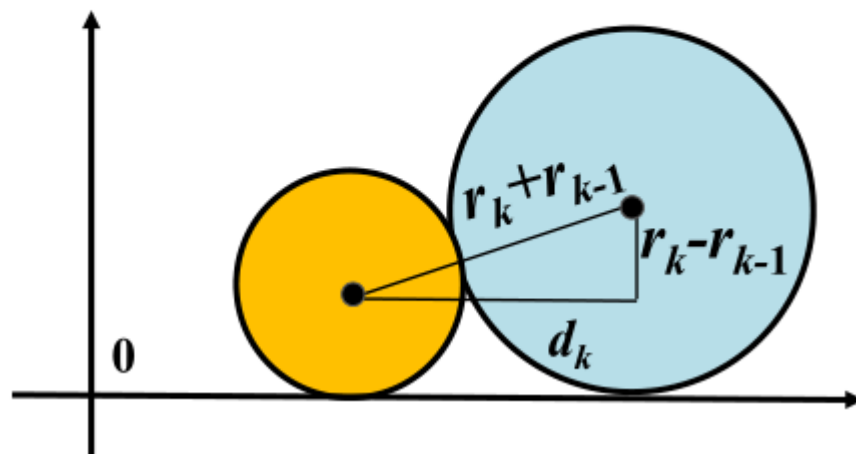
部分排列长度 $l_k = x_k + r_k + r_1$

排列长度 $\underline{l_n = x_k + d_{k+1} + d_{k+2} + \dots + d_n + r_n + r_1}$

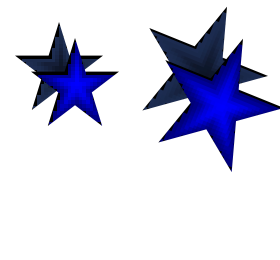


5.4 分支限界

有关量的计算



$$\begin{aligned} d_k &= \sqrt{(r_k + r_{k-1})^2 - (r_k - r_{k-1})^2} \\ &= 2\sqrt{r_{k-1}r_k} \end{aligned}$$



5.4 分支限界

代价函数

$$d_k = 2\sqrt{r_{k-1}r_k}$$

排列
长度

$$l_n = x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1$$

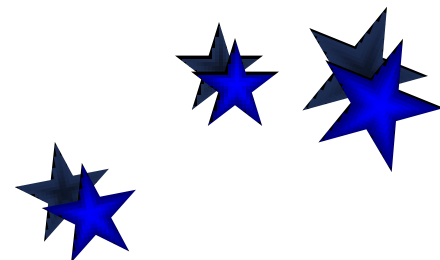
$$\geq x_k + 2(n-k)r + r + r_1$$

下界

$$L_k = x_k + (2n - 2k + 1)r + r_1$$

$$r = \min(r_{i_j}, r_k)$$

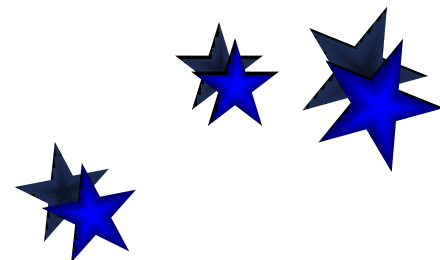
$$i_j \in \{1, 2, \dots, n\} - B$$



5.4 分支限界

时间复杂度

- 搜索树树叶数为 $O(n!)$
- 每条路径代价函数的计算为 $O(n)$
- 最坏情况下算法时间复杂度为
 $O(n \cdot n!) = O((n+1)!)$



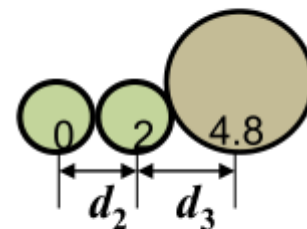
5.4 分支限界

实例

输入: $R=\{1,1,2,2,3,5\}$

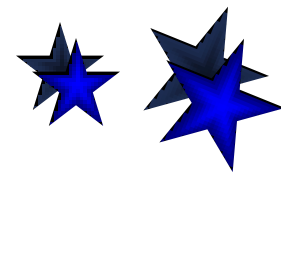
排列 $\langle 1,2,3,4,5,6 \rangle$

半径排列: 1,1,2,2,3,5



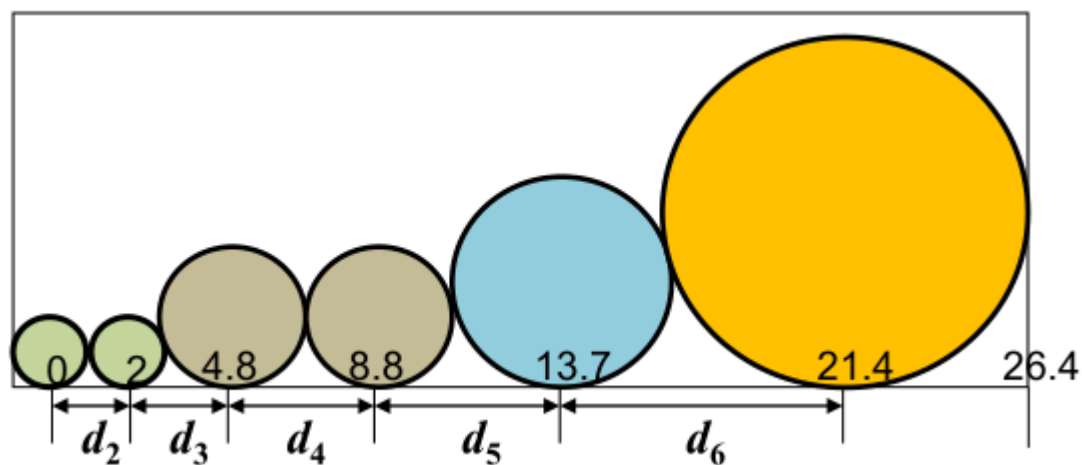
$$L_3 = 7.8 + 12 = 19.8$$

k	r_k	d_k	x_k	l_k	L_k
1	1	0	0	2	12
2	1	2	2	4	12
3	2	2.8	4.8	7.8	19.8
4	2	4	8.8	11.8	19.8
5	3	4.9	13.7	17.7	23.7
6	5	7.7	21.4	27.4	27.4



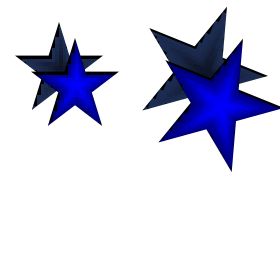
5.4 分支限界

实例计算



半径排列: 1,1,2,2,3,5, 可行解 $l_6=27.4$

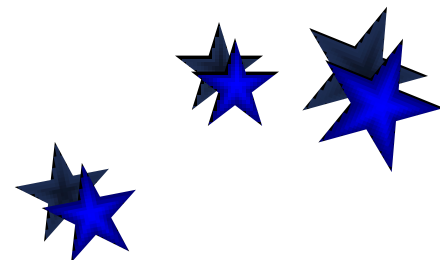
最优解: $\langle 1,3,5,6,4,2 \rangle$, 最短长度26.5



5.4 分支限界

小结

- 园排列问题的定义
- 如何设计代价函数?
已有的排列长度+后续的圆排列长度的下界
- 时间复杂度估计: $O((n+1)!)$



5.4 分支限界

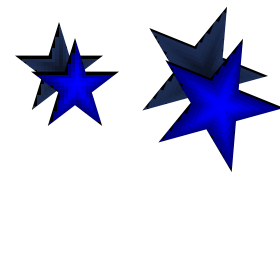
连续邮资问题

问题：给定 n 种不同面值的邮票，每个信封至多贴 m 张，试给出邮票的最佳设计，使得从 1 开始，增量为 1 的连续邮资区间达到最大？

实例： $n = 5, m = 4$.

设计1：面值 $X_1 = \langle 1, 3, 11, 15, 32 \rangle$,
✓ 邮资连续区间 $\{1, 2, \dots, 70\}$

设计2：面值 $X_2 = \langle 1, 6, 10, 20, 30 \rangle$,
邮资连续区间 $\{1, 2, 3, 4\}$



5.4 分支限界

算法设计

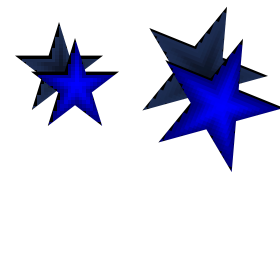
可行解 : $\langle x_1, x_2, \dots, x_n \rangle$, $x_1 = 1$,
 $x_1 < x_2 < \dots < x_n$

搜索策略: 深度优先

约束条件: 在结点 $\langle x_1, x_2, \dots, x_i \rangle$ 处,
邮资最大连续区间为 $\{1, \dots, r_i\}$, x_{i+1}
的取值范围是

$$\{x_i + 1, \dots, r_i + 1\}$$

若 $x_{i+1} > r_i + 1$, $r_i + 1$ 的邮资将没法支付.



5.4 分支限界

r_i 的计算

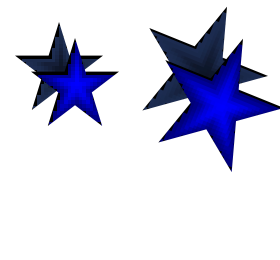
$y_i(j)$: 用至多 m 张面值 x_i 的邮票加上 $x_1, x_2, \dots, x_{i-1}$ 面值的邮票贴 j 邮资时的最少邮票数, 则

$$y_i(j) = \min_{1 \leq t \leq m} \{t + y_{i-1}(j - t x_i)\}$$

$$y_1(j) = j$$

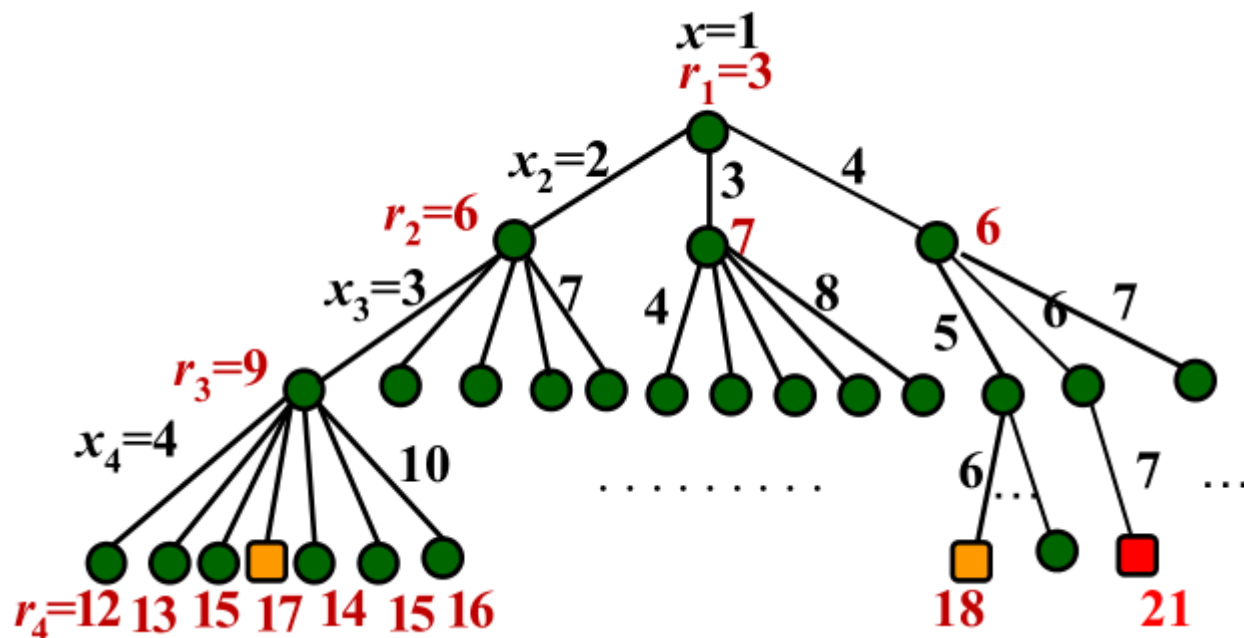
$$r_i = \min\{j \mid y_i(j) \leq m, y_i(j+1) > m\}$$

界: max , m 张邮票连续付的最大邮资



5.4 分支限界

部分搜索树 $n=4, m=3$



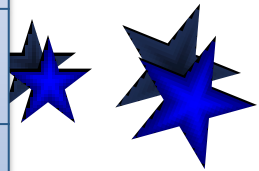
解: $X=\langle 1,4,6,7 \rangle$, 最大连续区间 $\{1, \dots, 21\}$

补充：求解任务分配问题

【问题描述】 有 n ($n \geq 1$) 个任务需要分配给 n 个人执行，每个任务只能分配给一个人，每个人只能执行一个任务。第 i 个人执行第 j 个任务的成本是 $c[i][j]$ ($1 \leq i, j \leq n$)。求出总成本最小的分配方案——**排列问题**。

4个人员、4个任务的信息

人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4



补充：求解任务分配问题

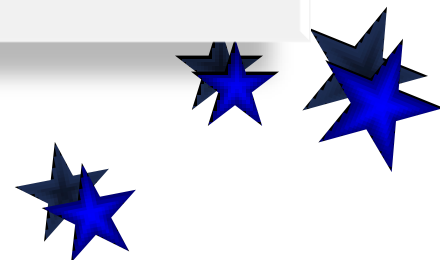
■ 实现方法1：回溯法实现——深度优先遍历实现
考虑为第*i*个人员分配任务（*i*从1开始），由于每个任务只能分配给一个人员，为了避免重复分配，设计一个worker布尔数组，初始时均为false，当任务*j*分配后置worker[*j*]=true。求解结果表示如下：

```
int x[MAXN];           //临时解
int cost=0;            //临时解的成本
int bestx[MAXN];       //最优解
int mincost=INF;       //最优解的成本
bool worker[MAXN];     //worker[j]表示任务j是否已经分配
```



补充：求解任务分配问题

```
void dfs(int i)           //为第i个人员分配任务
{ if (i>n)                //到达叶子结点
{ if (cost<mincost)       //比较临时解和当前最优解求最优解
{ mincost=cost;
  for (int j=1;j<=n;j++)
    bestx[j]=x[j];
  }
}
```



补充：求解任务分配问题

```
else {  
    for (int j=1;j<=n;j++)  
        if (!worker[j])  
            {worker[j]=1;  
             x[i]=j;           //任务j分配给人员i  
             cost+=c[i][j];  
             dfs(i+1);        //为人员i+1分配任务  
             worker[j]=false; //回退  
             x[i]=0;  
             cost-=c[i][j];  
            }  
    } //end if  
} //end dfs
```



补充：求解任务分配问题

■ 实现方法2：分支限界法—广度优先搜索

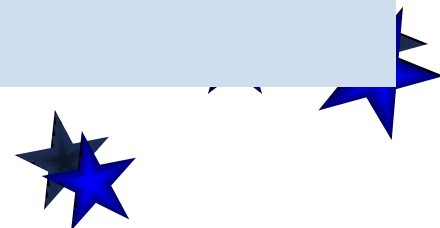
② 下界限界函数设计

lb为当前结点对应分配方案的成本下界。

例如搜索树中某结点 e 对应部分解： $x[]=[2, 1, 0, 0]$ ，表示第1个人员分配任务2，第2个人员分配任务1，第3、4个人员没有分配任务；

相对应有 $worker[]=[T, T, F, F]$ ，表示任务1和2已经分配，而任务3、4还没有分配。此时 e 结点的成本值是：

$e.cost=c[1][2]+c[2][1]=2+6=8。$



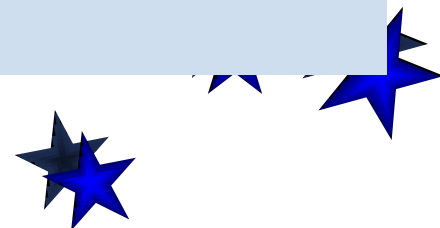
补充：求解任务分配问题

人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4

e

```
no=**  
i=2  
x={2,1,0,0}  
worker={T,T,F,F}  
cost=8  
lb=13
```

下一步最好的情况是在数组 c 中第3行和第4行中找到非第1、2列（因为任务1、2已经分配）中最小元素和，显然为 $1+4=5$ ，即其 $e.lb=e.cost+5=13$ 。



补充：求解任务分配问题

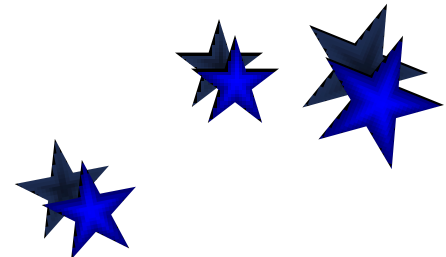
1

人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4

```
no=1  
i=0  
x[]={0,0,0,0}  
Worker={F,F,F,F}  
cost=0  
Lb=10
```

根结点，初始状态

因为 $e.i=0$ ，因此从 $e.i+1$ ，即第1行开始到第n行，从每行中找出最小值累加得到根结点的lb值（10）。



人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4

广度优先搜索：用队列实现
按层遍历

1

no=1
i=1, cost=0, lb=10
x[]={0,0,0,0}
Worker={F,F,F,F}

j=1

j=2

j=3

j=4

为人员1分配
任务j

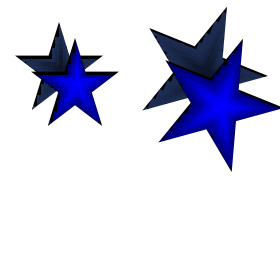
no=2
i=1, cost=9
lb=17
x[]={1,0,0,0}
worker={T,F,F,F}

no=3
i=1, cost=2
lb=10
x[]={2,0,0,0}
worker={F,T,F,F}

no=4
i=1, cost=7
lb=20
x[]={3,0,0,0}
worker={F,F,T,F}

no=5
i=1, cost=8
lb=18
x[]={4,0,0,0}
worker={F,F,F,T}

从e.i+1行到第n行，去掉worker[j]=T的列，
找出每行最小值进行累加。



人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4

广度优先搜索：用队列实现 按层遍历

1

5

no=2
i=1, cost=9
lb=17
x[]={1,0,0,0}
worker={T,F,F,F}

no=3
i=1, cost=2
lb=10
x[]={2,0,0,0}
worker={F,T,F,F}

no=4
i=1, cost=7
lb=20
x[]={3,0,0,0}
worker={F,F,T,F}

no=5
i=1, cost=8
lb=18
x[]={4,0,0,0}
worker={F,F,F,T}

j=2

j=3

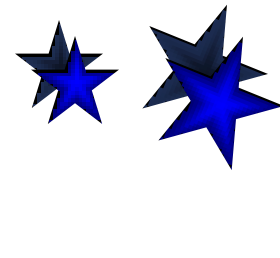
j=4

为人员2分配
任务j

no=6
i=2, cost=13
lb=18
x[]={1,2,0,0}
worker={T,T,F,F}

no=7
i=2, cost=12
lb=24
x[]={1,3,0,0}
worker={T,F,T,F}

no=8
i=2, cost=16
lb=23
x[]={1,4,0,0}
worker={T,F,F,T}



结点e:

no=

i=2

cost=8

lb=13

x[]={2,1,0,0}

worker={T,T,F,F}

人员	任务1	任务2	任务3	任务4
1	9	2	7	8
2	6	4	3	7
3	5	8	1	8
4	7	6	9	4

void bound(NodeType &e)

//求结点e的限界值

{ int minsum=0;

for (int i=e.i+1;i<=n;i++) //求c[e.i+1..n]行中最小元素和

{ int minc=INF;

for (int j=1;j<=n;j++) //各列中仅仅考虑没有分配的任务

if (e.worker[j]==false && c[i][j]<minc)

minc=c[i][j];

minsum+=minc;

}

e.lb=e.cost+minsum;

}

补充：求解任务分配问题

```
void bfs()                                //求解任务分配
{
    int j;
    NodeType e,e1;
    queue qu;
    memset(e.x,0,sizeof(e.x));           //初始化根结点的x
    memset(e.worker,0,sizeof(e.worker)); //初始化根结点的worker
    e.i=0;                                //根结点，指定人员为0
    e.cost=0;
    bound(e);                             //求根结点的lb
    e.no=total++;
    qu.push(e);                           //根结点进队列
}
```


补充：求解任务分配问题

```
while (!qu.empty())
{
    e=qu.top(); qu.pop();           //出队结点e，当前考虑人员e.i
    if (e.i==n)                    //达到叶子结点
        if (e.cost<mincost)        //比较求最优解
        {
            mincost=e.cost;
            for (j=1;j<=n;j++)
                bestx[j]=e.x[j];
        }
    else //不是叶子结点，则扩展当前结点e
    {
```



```

e1.i=e.i+1;      //扩展分配下一个人员的任务, 对应结点e1
for (j=1;j<=n;j++)      //考虑n个任务
{ if (e.worker[j]) //任务j是否已分配人员,若已分配, 跳过
    continue;
    for (int i1=1;i1<=n;i1++)      //复制e.x得到e1.x
        e1.x[i1]=e.x[i1];
    e1.x[e1.i]=j;      //为人员e1.i分配任务j
    for (int i2=1;i2<=n;i2++)      //复制e.worker得到e1.worker
        e1.worker[i2]=e.worker[i2];
    e1.worker[j]=true;      //表示任务j已经分配
    e1.cost=e.cost+c[e1.i][j];
    bound(e1);      //求e1的lb
    e1.no=total++;
    if (e1.lb<mincost)      //剪枝
        qu.push(e1);
    }
}
}

```

- 使用**优先队列**实现：队列中的结点按照**代价值lb**从小到大排序，扩展队头结点，即优先扩展最有利于找到最优解的分支

